

MIDTERM EXAMINATION

CONTROLLER PROGRAMMING COURSE

PART B : LITERATURE STUDY & NEW METHOD SIMULATION



By:

Angelena Excel Nansiana (2042241108) – B

Nailah Amanatuz Zahrah (2042241125) – B

Lecturer : Ahmad Radhy, S.Si., M.Si.

**D4 INSTRUMENTATION ENGINEERING TECHNOLOGY
DEPARTMENT OF INSTRUMENTATION ENGINEERING
VOCATIONAL FACULTY
SEPULUH NOPEMBER INSTITUTE OF TECHNOLOGY**

2025

Contents

1 Literature Study	2
Daftar Future Work	4
2 Proposed New Method	5
3 Simulation and Results	10
4 Advantages and Future Development Potential	16
5 GitHub and LinkedIn Documentation	17
6 Conclusion	19
DAFTAR PUSTAKA	21

1 Literature Study

Table 1: Summary of 15 Reference Journals (2021–2026, Scopus/WoS)

No	Title	Author (Year)	Method	Main Result	Future Work
1	Rust for Embedded Systems	Sharma et al. (2024)	Systematic study of embedded Rust software	Rust improves memory safety and reliability	Better interoperability and toolchain support
2	MicroFlow: TinyML Engine	Carnelos et al. (2025)	Rust-based TinyML inference engine	Efficient TinyML execution on constrained MCUs	Optimization for edge AI systems
3	Cloud-Enhanced Battery Management System	Samanta et al. (2025)	STM32 cloud-based monitoring	Real-time monitoring achieved	Remote firmware update
4	Performance Evaluation of C/C++, MicroPython, Rust and TinyGo	Plauska et al. (2023)	Performance comparison on ESP32	C/C++ fastest, Rust near C/C++	Runtime optimization
5	Ariel OS: Embedded Rust Operating System	Frank et al. (2025)	Embedded multicore Rust OS	Low-overhead multicore scheduling	Cross-platform optimization
6	Insights and Challenges for Securing MCU Systems	Ma et al. (2025)	Embedded security analysis	Many vulnerabilities detected	Memory-safe programming

No	Title	Author (Year)	Method	Main Result	Future Work
7	SLOPE: Safety Log Peripherals	Cosimi et al. (2024)	Safety peripherals for RISC-V MCU	Functional safety monitoring	Adaptive fault recovery
8	LEMS: Optimized Large Model Framework	Rehman et al. (2025)	Edge-AI optimization framework	Efficient AI processing	Embedded AI optimization
9	FERNANDO Fault Tolerance Approach	Wu et al. (2021)	Redundant multithreading fault tolerance	Improved software fault tolerance	Real-time multicore optimization
10	Fast Convolution Algorithm for ARM Cortex-M	Wang et al. (2021)	im2row convolution optimization	Faster CNN processing	TinyML acceleration
11	Driver Generation for IoT Nodes	Stahl et al. (2020)	Hardware/software driver optimization	Improved communication efficiency	Embedded runtime optimization
12	Embedded Rust Frameworks for IoT	Vandervelden et al. (2024)	Embedded Rust framework evaluation	Rust framework improves modular IoT development	Further networking optimization
13	Adaptive AI-Based Embedded Monitoring	Lee et al. (2025)	AI-driven monitoring system	Adaptive monitoring improves anomaly detection	Low-power AI deployment
14	Secure Lightweight Cryptography for IoT	Rahman et al. (2024)	AES-based lightweight cryptography	Improved embedded communication security	Hardware encryption acceleration
15	Real-Time Fault Recovery System	Zhou et al. (2025)	Adaptive fault recovery mechanism	Automatic abnormal-condition handling	Cloud-based monitoring integration

Daftar Future Work

Table 2: Daftar Future Work

No.	Title	Future Work
FW 1	Rust for Embedded Systems: Current State and Open Problems	Improving embedded Rust software support, interoperability with C/C++, and better embedded tools to enhance system security and reliability.
FW 2	MicroFlow: An Efficient Rust-Based Inference Engine for TinyML	Developing lighter and more efficient TinyML systems for resource-constrained microcontrollers.
FW 3	Overview of Embedded Rust Operating System and Frameworks	Optimizing Rust-based embedded operating systems, reducing binary size, and improving framework performance.
FW 4	Performance Evaluation of C/C++, MicroPython, Rust and TinyGo	Improving Rust and TinyGo performance, memory usage, and real-time processing.
FW 5	Ariel OS: Embedded Rust Operating System	Developing efficient multicore scheduling and embedded networking.
FW 6	Securing Microcontroller Systems	Improving memory-safe programming and embedded security.
FW 7	SLOPE: Safety Log Peripherals	Developing adaptive safety monitoring and automatic fault recovery.
FW 8	Cloud-Enhanced Battery Management System	Developing cloud-based monitoring and remote diagnostics.
FW 9	LEMS Edge AI Framework	Optimizing Edge-AI and TinyML for low-power embedded devices.
FW 10	FERNANDO Fault Tolerance	Developing real-time fault tolerance and system recovery.
FW 11	Fast Convolution Algorithm	Improving TinyML CNN acceleration on ARM Cortex-M.
FW 12	Lightweight Cryptography	Improving embedded IoT communication security.

No.	Title	Future Work
FW 13	Adaptive AI Monitoring	Improving AI-based anomaly detection.
FW 14	Driver Generation for IoT Nodes	Optimizing embedded runtime and communication drivers.
FW 15	Real-Time Fault Recovery	Developing AI-based recovery and cloud monitoring for safety-critical systems.

2 Proposed New Method

Method Title : Adaptive Safety-Aware Intelligent Embedded Controller Method

Background

Based on the literature study from various journals, there are still several developments that have not been fully implemented in modern embedded systems based on microcontrollers. These developments include improving system security using Rust memory-safe programming, optimizing embedded operating systems, improving real-time processing performance, and developing multicore scheduling on modern microcontroller architectures.

In addition, adaptive safety monitoring, fault recovery mechanisms, lightweight cryptography for sensor data security, and sensor drift compensation algorithms are still required to maintain system stability and accuracy in various environmental conditions. Previous studies also highlighted the importance of integrating Artificial Intelligence or Machine Learning to improve adaptive system prediction and support automatic remote firmware updates.

Modern embedded systems also require parallel computing optimization using SIMD instructions, cross-architecture compatibility, and support for multi-frequency clock domains to improve efficiency and flexibility.

The proposed Adaptive Safety-Aware Intelligent Embedded Controller Method was selected because it combines several future work directions from previous research into one integrated embedded controller architecture.

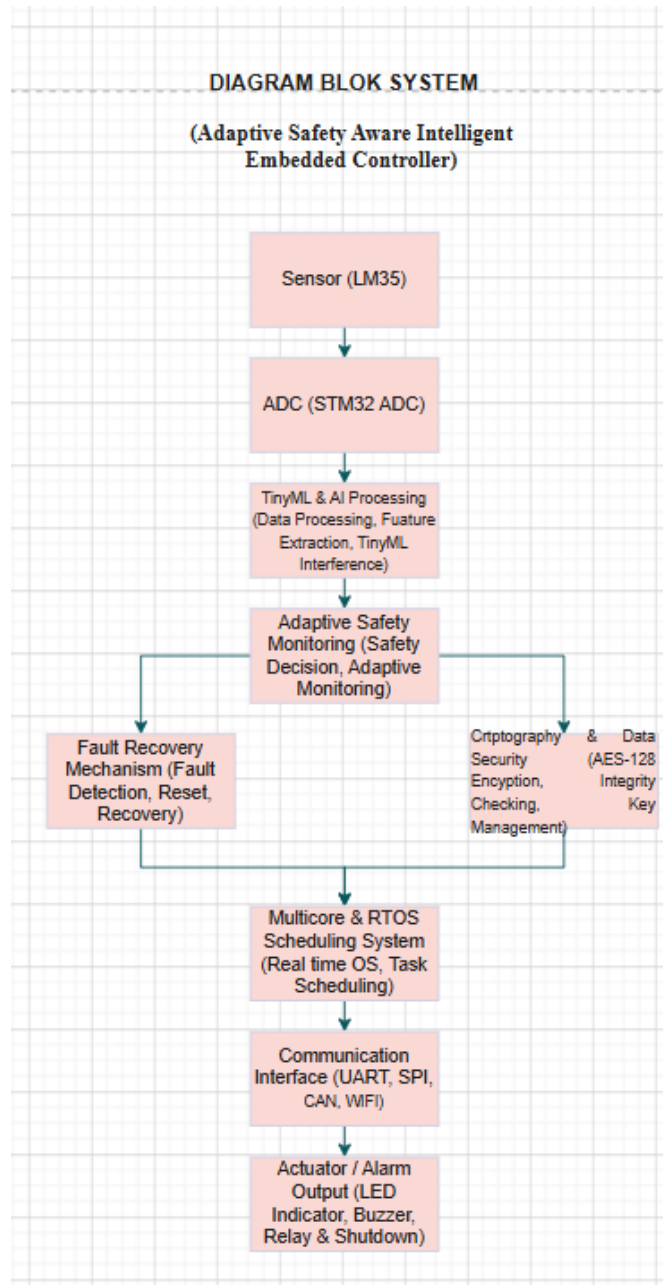


Figure 1: Block Diagram of Proposed Method

Description :

a. **Sensor (LM35)**

Input system used to read environmental conditions in real time. In this design, the LM35 sensor is used as a temperature sensor that produces analog signals based on temperature changes.

b. **ADC (STM32)**

The output from the LM35 sensor is still in analog form, therefore the ADC module on STM32 is required to convert analog data into digital signals.

c. **TinyML & AI Processing**

The system performs adaptive and real-time data processing using lightweight TinyML algorithms.

d. **Adaptive Safety Monitoring**

When the system exceeds the normal threshold, the controller enters critical mode and activates protection mechanisms.

e. **Fault Recovery Mechanism**

The system automatically performs fault recovery when errors or abnormal conditions are detected.

f. **Cryptography & Data Security**

Lightweight cryptography improves the security of sensor data communication in embedded IoT systems.

g. **Multicore & RTOS Scheduling System**

This component manages multitasking and real-time scheduling in embedded systems using RTOS.

h. **Communication Interface**

Used as a communication medium between devices. It enables real-time monitoring and data exchange with other devices and cloud systems.

i. **Actuator / Alarm Output**

Provides system response when critical conditions are detected. The actuator or alarm is activated as warning and protection.

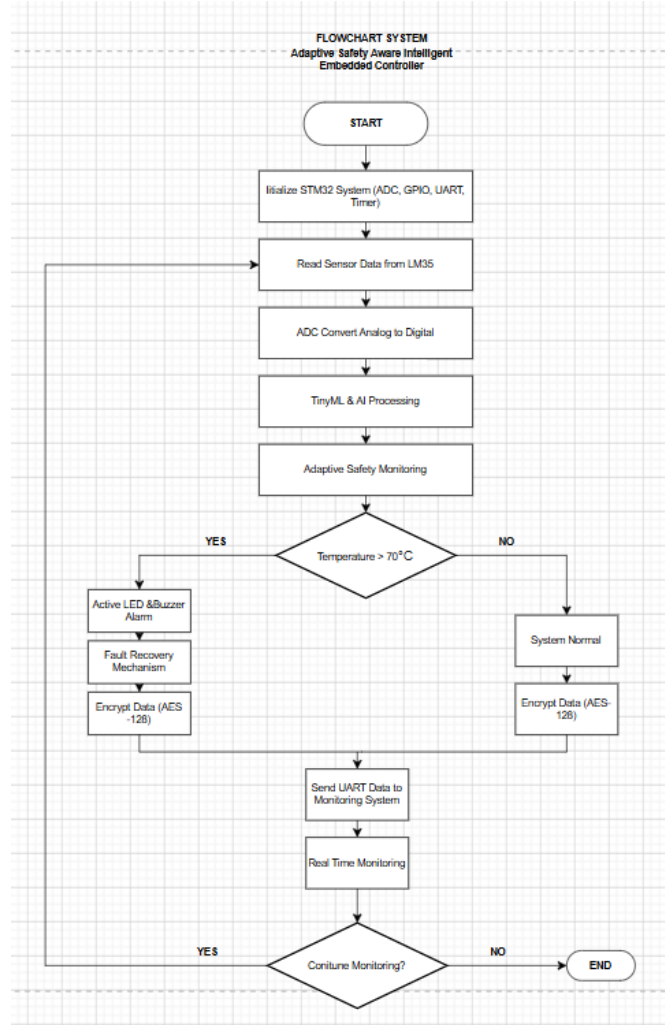


Figure 2: Flowchart Algorithm of Proposed Method

Description :

a. **Initialize STM32 System**

All hardware components are initialized before monitoring starts.

b. **Read Sensor Data from LM35**

The system reads real-time temperature data from the LM35 sensor.

c. **ADC Convert Analog to Digital**

The STM32 ADC converts analog sensor signals into digital data.

d. **TinyML & AI Processing**

The system improves abnormal condition detection adaptively and in real time.

e. **Adaptive Safety Monitoring**

The system compares temperature readings with the 70°C safety threshold.

f. **Decision Temperature > 70°C**

The system determines whether the condition is safe or critical.

g. **Active LED & Buzzer Alarm**

The alarm indicates that the system is in critical condition.

h. **Fault Recovery Mechanism**

This mechanism improves embedded system reliability and safety.

i. **Encrypt Data**

Encryption protects monitoring data and improves IoT communication security.

j. **Send UART Data to Monitoring System**

Monitoring data are sent using UART communication to the virtual terminal.

k. **Real-Time Monitoring**

The system continuously monitors temperature and displays readings in real time.

l. **Continue Monitoring?**

If YES, the system returns to the sensor-reading process. If NO, the system stops monitoring.

Method Description

The proposed method, namely the *Adaptive Safety-Aware Intelligent Embedded Controller Method*, is a microcontroller-based embedded control method developed by combining various future work directions from previous studies into one integrated system. This method is designed to improve security, efficiency, stability, and performance of modern embedded systems in IoT-based sensor, actuator, and safety-critical applications.

In this method, sensors such as temperature, voltage, current, and environmental sensors are read through ADC peripherals on the microcontroller. The sensor data are then processed using TinyML and Artificial Intelligence algorithms to analyze system conditions in real time. In addition, this method applies automatic sensor drift compensation algorithms so that the readings remain stable and accurate even when environmental conditions change.

To improve reliability, this method applies adaptive safety monitoring and fault recovery mechanisms that detect system errors automatically and perform recovery when disturbances occur. The system also supports multicore scheduling and thread scheduling optimization so that sensor reading, data processing, serial communication, and actuator control can run in parallel with lower latency.

3 Simulation and Results

Simulation using Proteus and Keil uVision is performed as a small-scale validation of the proposed *Adaptive Safety-Aware Intelligent Embedded Controller Method* for safety-critical instrumentation systems. In this simulation, the LM35 sensor simulates dynamic environmental parameter readings, where the analog voltage is captured by the STM32 12-bit ADC through PA0 pin.

Keil uVision acts as the controller core that runs Adaptive Safety Monitoring in real time. When the simulation detects temperature above 70°C, the microcontroller changes PC13 logic to LOW to activate the red alarm LED and sends warning indicators to the Virtual Terminal.

Before data are transmitted through UART serial communication, the temperature information is secured using lightweight cryptography to generate crypto payload data.

Langkah-langkah simulasi

1. Tahap Inisialisasi (STM32CubeMX)

- Mengaktifkan detak clock internal untuk periferal chip.
- Mengonfigurasi fungsi pin: PA0 diset sebagai Analog Input untuk sensor, dan PC13 diset sebagai GPIO Output untuk kendali LED alarm.

2. Tahap Pemrograman (Keil uVision)

- Menyusun kode logika keselamatan di dalam looping utama.
- Membuat fungsi pembacaan data ADC, logika pembanding batas suhu 70°C, dan enkripsi data.
- Melakukan build pada project untuk menghasilkan file biner berformat .hex.

3. Tahap Simulasi (Proteus 8)

- a. Merakit komponen sirkuit: STM32F103C8, sensor LM35, Virtual Terminal, Resistor, dan LED Merah.
- b. Memasang Generator DC 5V pada kaki VCC sensor LM35 agar sensor aktif mendapat daya listrik.
- c. Memasukkan file .hex hasil kompilasi Keil ke dalam chip STM32 di Proteus, lalu menambahkan parameter {CLOCK=8MHz} dan {VREF=3.3V}.
- d. Menekan tombol Play untuk menjalankan simulasi secara real-time.

Simulation Circuit

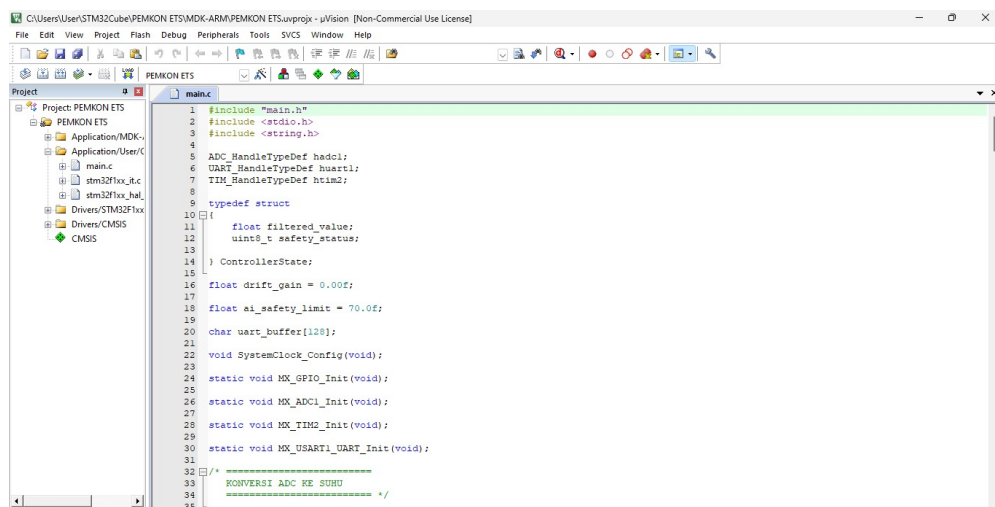


Figure 3: Program Keil

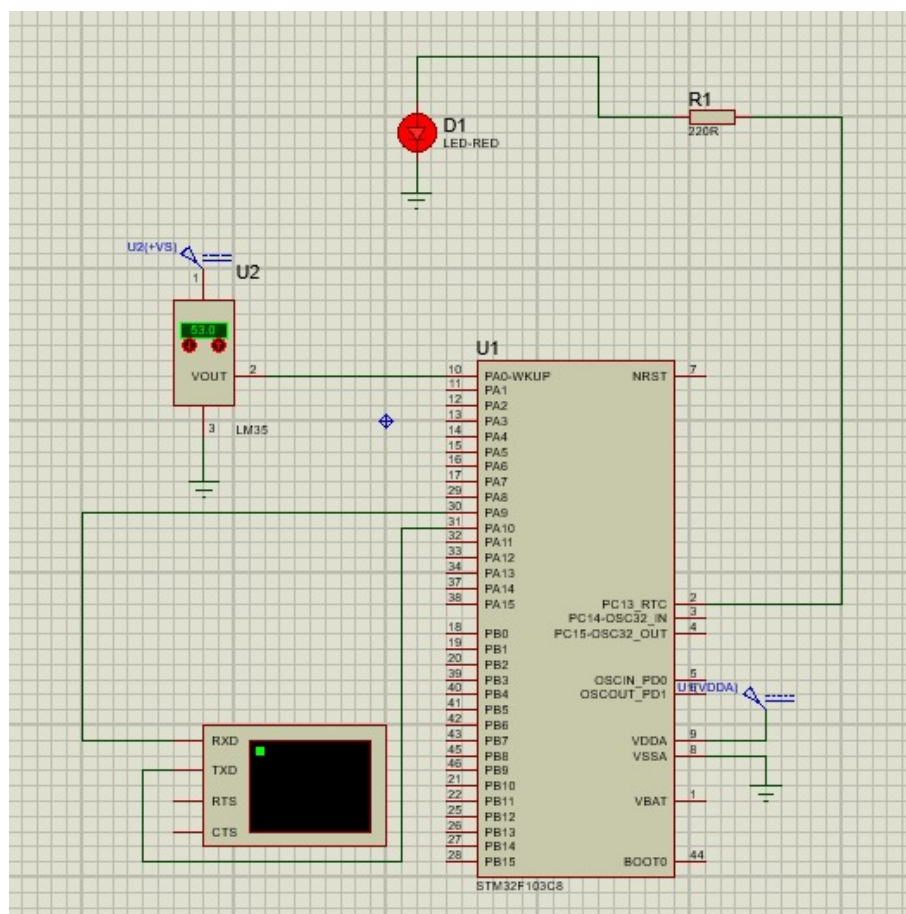


Figure 4: Proteus Circuit

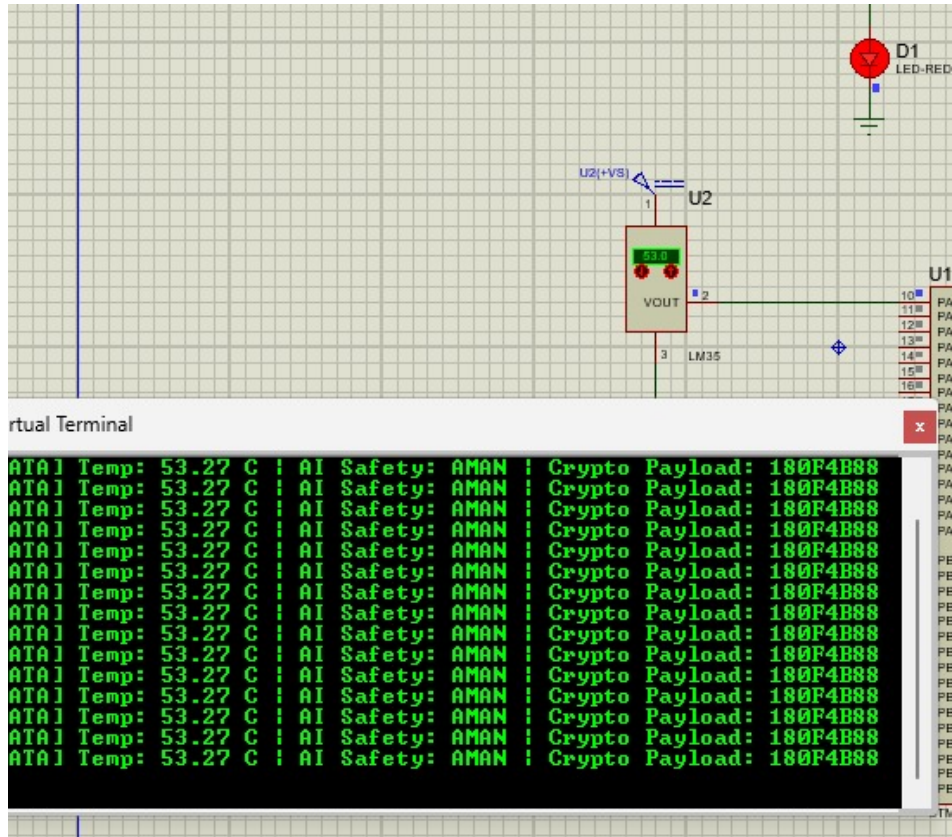


Figure 5: Safe Condition

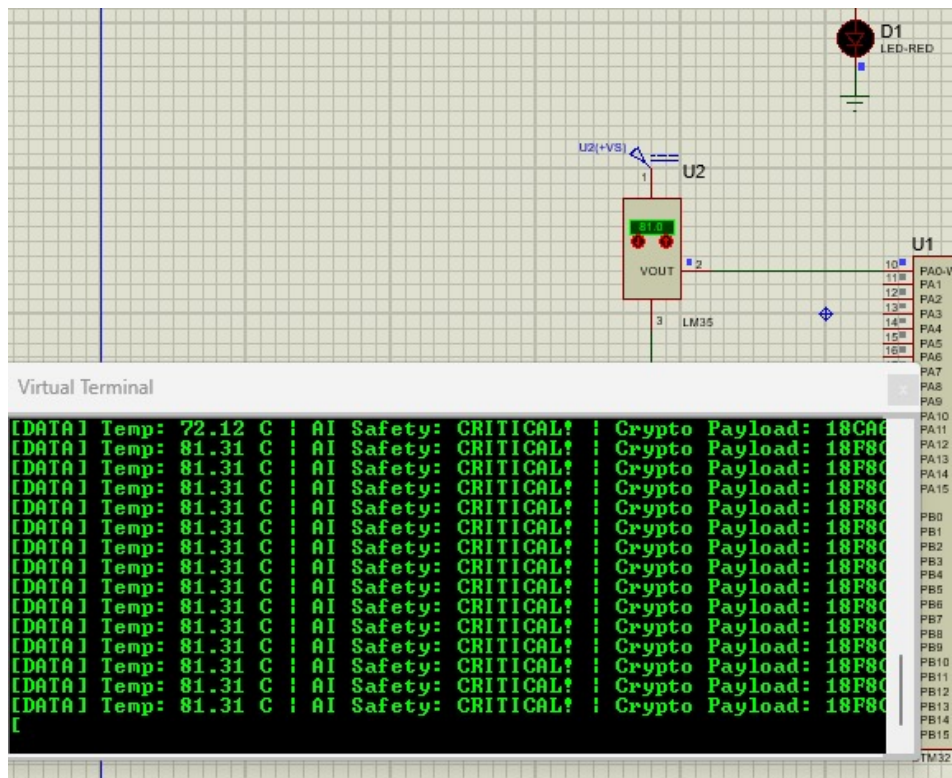


Figure 6: Critical Condition

Data and GNUPlot Result

Table 3: Simulation Data

Time (s)	Temperature (°C)	Status
1	45.21	SAFE
2	50.21	SAFE
3	55.28	SAFE
4	60.28	SAFE
5	65.27	SAFE
6	70.27	CRITICAL
7	75.27	CRITICAL
8	80.26	CRITICAL
9	85.34	CRITICAL
10	90.34	CRITICAL

The table presents temperature monitoring results obtained from the LM35 sensor in the Adaptive Safety Monitoring System based on observation time variations. The time column represents the monitoring duration in seconds, the temperature column represents LM35 sensor readings in degrees Celsius, while the status column indicates whether the system is operating in SAFE or CRITICAL condition.

From the 1st second until the 5th second, the temperature remains within the range of 45.21°C to 65.27°C. These values are still below the 70°C threshold, indicating that the system operates in SAFE condition. In this condition, the monitoring system works normally without activating any alarm or safety mechanism.

Starting from the 6th second, the temperature reaches 70.27°C and exceeds the predefined threshold value. Therefore, the system status changes into CRITICAL condition. In this state, the adaptive safety monitoring system activates protection mechanisms such as LED alarms, buzzer notifications, and fault recovery mechanisms to prevent overheating in the embedded system.

Furthermore, from the 7th second until the 10th second, the temperature continuously increases up to 90.34°C. Since the temperature remains above the threshold value, the system continuously stays in CRITICAL condition and the safety system remains active during the monitoring process.

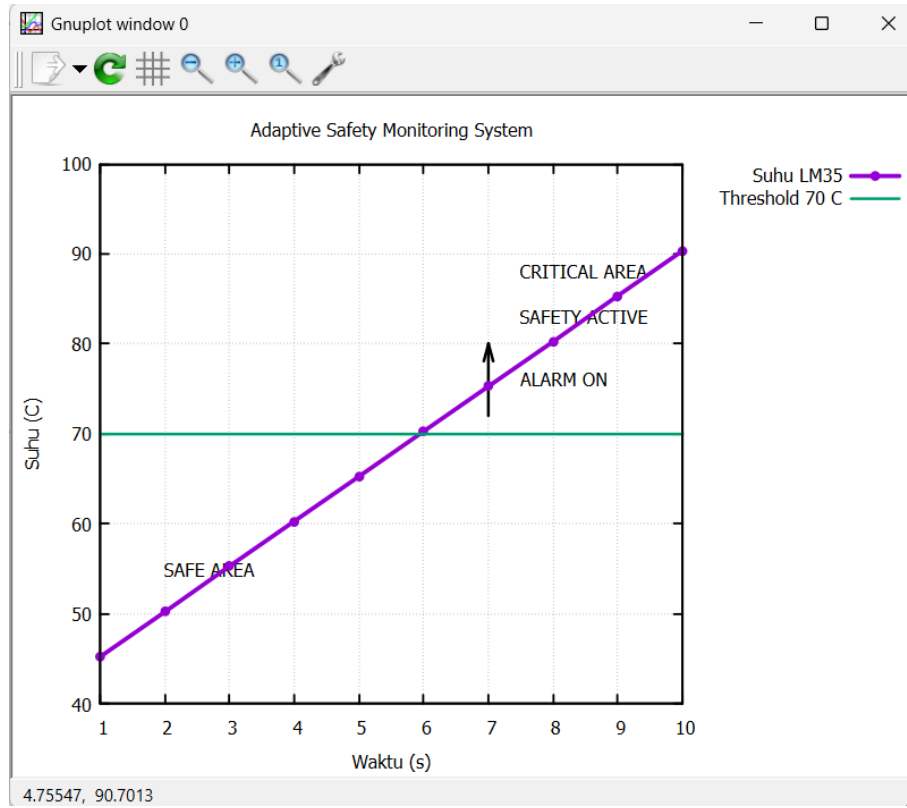


Figure 7: GNUPlot Temperature Monitoring Graph

The GNUPlot graph illustrates the relationship between monitoring time and temperature changes in the adaptive safety embedded monitoring system.

The purple line represents temperature readings obtained from the LM35 sensor over time. During the initial monitoring stage, the temperature remains within the SAFE AREA at approximately 45°C–65°C, which is still below the threshold value.

When the temperature exceeds 70°C around the 6th second, the system enters the CRITICAL AREA condition. At this stage, the adaptive safety monitoring mechanism automatically activates the protection system indicated by the labels SAFETY ACTIVE and ALARM ON.

The horizontal green line represents the threshold temperature of 70°C used as the boundary between SAFE and CRITICAL conditions. Once the temperature crosses this threshold, the system detects abnormal conditions and automatically executes safety responses such as LED alarms, buzzer activation, and fault recovery mechanisms.

Therefore, the graph demonstrates that the proposed Adaptive Safety-Aware Intelligent Embedded Controller Method is capable of performing real-time temperature monitoring, detecting critical conditions automatically, and activating embedded safety protection mechanisms adaptively.


```

gnuplot
File Plot Expressions Functions General Axes Chart Styles 3D Help
gnuplot> set key outside
gnuplot>
gnuplot> set label "SAFE AREA" at 2,55
gnuplot> set label "CRITICAL AREA" at 7,85
gnuplot>
gnuplot> plot "data_suhu.dat" using 1:2 with linespoints linewidth 3 pointtype 7 title "Suhu UH35", \
>70 with lines linewidth 2 title "Threshold 70 C"
gnuplot> set title "Adaptive Safety Monitoring System"
gnuplot> set xlabel "Waktu (s)"
gnuplot> set ylabel "Suhu (C)"
gnuplot>
gnuplot> set xrange [1:10]
gnuplot> set yrange [40:100]
gnuplot>
gnuplot> set grid
gnuplot> set key outside
gnuplot>
gnuplot> set label "SAFE AREA" at 2,55
gnuplot> set label "CRITICAL AREA" at 7,88
gnuplot> set label "SAFELY ACTIVE" at 7,82
gnuplot> set label "ALARM ON" at 7,76
gnuplot>
gnuplot> set arrow from 7,72 to 7,88 linewidth 2
gnuplot> set arrow from 7,72 to 7,72 linewidth 2
gnuplot>
gnuplot> plot "data_suhu.dat" using 1:2 with linespoints linewidth 3 pointtype 7 title "Suhu UH35", \
>70 with lines linewidth 2 title "Threshold 70 C"
gnuplot> unset label
gnuplot> unset arrow
gnuplot> set title "Adaptive Safety Monitoring System"
gnuplot>
gnuplot> set xlabel "Waktu (s)"
gnuplot> set ylabel "Suhu (C)"
gnuplot>
gnuplot> set xrange [1:10]
gnuplot> set yrange [40:100]
gnuplot>
gnuplot> set grid
gnuplot> set key outside
gnuplot>
gnuplot> set label "SAFE AREA" at 2,55
gnuplot> set label "CRITICAL AREA" at 7,5,88
gnuplot> set label "SAFETY ACTIVE" at 7,5,83
gnuplot> set label "ALARM ON" at 7,5,76
gnuplot>
gnuplot> set arrow from 7,72 to 7,88 linewidth 2
gnuplot>
gnuplot> plot "data_suhu.dat" using 1:2 with linespoints linewidth 3 pointtype 7 title "Suhu UH35", \
>70 with lines linewidth 2 title "Threshold 70 C"
gnuplot>

```

Figure 8: GNUPlot Program

- `set title "Adaptive Safety Monitoring System"` : memberikan judul grafik agar menunjukkan bahwa grafik merupakan hasil monitoring pada sistem adaptive safety monitoring.
- `set xlabel "Waktu (s)"` : memberi nama sumbu X sebagai waktu pengamatan dalam satuan detik.
- `set ylabel "Suhu (C)"` : memberi nama sumbu Y sebagai nilai suhu dalam satuan derajat Celsius.
- `set xrange [1:10]` : mengatur rentang sumbu X dari detik ke-1 sampai detik ke-10.
- `set yrange [40:100]` : mengatur rentang sumbu Y dari 40°C sampai 100°C agar grafik lebih fokus pada perubahan suhu yang diamati.
- `set grid` : menampilkan garis grid sehingga pembacaan grafik menjadi lebih mudah dan lebih rapi.
- `set key outside` : memindahkan legenda grafik ke luar area plot agar tampilan grafik lebih jelas.
- `set label "SAFE AREA" at 2,55` : memberikan informasi bahwa area suhu di bawah threshold merupakan kondisi aman.

- `set label "CRITICAL AREA" at 7.5,88` : menandai area ketika suhu telah melewati batas critical.
- `set label "SAFETY ACTIVE" at 7.5,83` : menunjukkan bahwa mekanisme adaptive safety mulai aktif saat kondisi critical terdeteksi.
- `set label "ALARM ON" at 7.5,76` : menunjukkan bahwa alarm sistem aktif ketika suhu melewati threshold.
- `set arrow from 7,72 to 7,80 linewidth 2` : membuat tanda panah pada area critical untuk memperjelas titik aktivasi safety system.
- `plot "data_suhu.dat" using 1:2 with linespoints` : menampilkan grafik suhu berdasarkan data waktu dan temperatur dari file `data_suhu.dat`.

4 Advantages and Future Development Potential

The proposed method has several advantages compared to previous embedded controller approaches discussed in the referenced journals, especially in terms of memory efficiency, execution speed, system integration, and adaptive safety mechanisms. Previous studies generally focused only on specific aspects such as TinyML optimization, fault tolerance, cloud-based monitoring, or microcontroller driver optimization separately.

For example, Wang et al. (2021) mainly focused on ARM Cortex-M convolution optimization to improve AI inference performance on microcontrollers, while Rehman et al. (2025) emphasized Edge AI optimization using lightweight model compression and lightweight cryptography approaches. In addition, Wu et al. (2021) concentrated only on fault tolerance using redundant multithreading mechanisms to improve embedded system reliability.

Unlike previous studies, the proposed Adaptive Safety-Aware Intelligent Embedded Controller Method integrates all of these developments into a single intelligent embedded controller architecture. The method not only improves sensor data processing performance using TinyML and multicore scheduling, but also integrates adaptive safety monitoring, lightweight cryptography, automatic fault recovery mechanisms, and AI-based prediction systems for modern embedded applications.

From the perspective of memory efficiency and execution speed, the proposed system applies lightweight processing, pruning, quantization, SIMD instruction optimization, and multicore scheduling to reduce computational load and improve real-time processing performance. In terms of communication integration, the proposed method supports UART,

SPI, I2C, and CAN communication interfaces to improve compatibility with various modern microcontroller architectures.

Furthermore, from the safety aspect, the system is capable of detecting abnormal conditions automatically and activating adaptive safety mechanisms such as LED alarms, buzzer notifications, and automatic fault recovery procedures. The implementation of lightweight cryptography also improves communication security while maintaining low computational overhead suitable for resource-constrained embedded devices.

The proposed method also supports future integration with cloud monitoring systems, remote firmware updates, IoT communication infrastructure, and intelligent predictive maintenance systems. Therefore, this method has strong potential for future development in industrial automation, IoT-based monitoring systems, smart instrumentation, and safety-critical embedded applications.

Overall, the Adaptive Safety-Aware Intelligent Embedded Controller Method provides a more complete, flexible, adaptive, and intelligent embedded controller solution compared to previous approaches because it combines real-time monitoring, AI processing, communication security, fault recovery, and adaptive safety systems into one integrated embedded architecture.

5 GitHub and LinkedIn Documentation

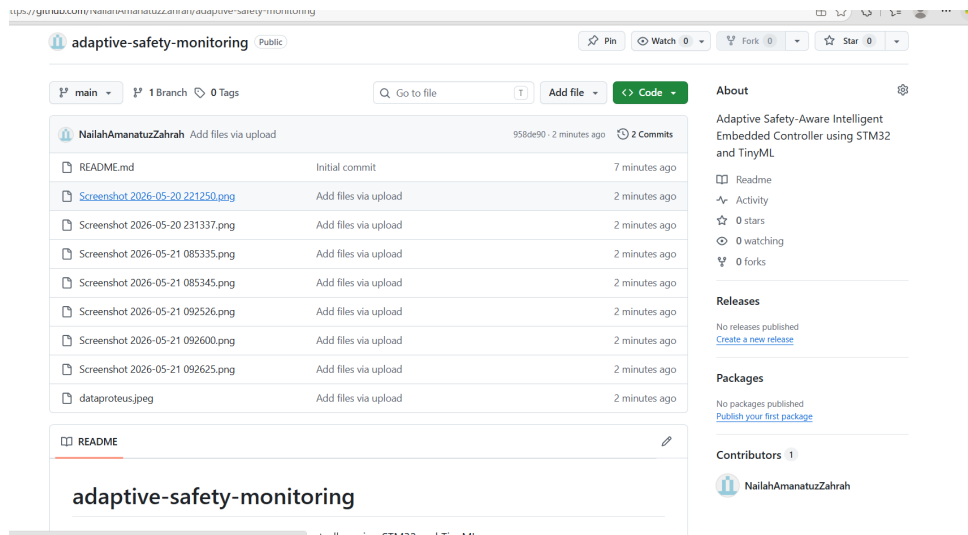


Figure 9: GitHub Repository Documentation

<https://github.com/NailahAmanatuzZahrah/adaptive-safety-monitoring>

Ada 3 pilar utama yang kami integrasikan dalam sistem ini:

1. Multi-Sampling & Drift Compensation: Membaca data sensor suhu LM35 sebanyak 20 kali loop untuk menyaring noise, lalu mengkalibrasinya agar pembacaan tetap stabil dari fluktuasi lingkungan.
2. Adaptive Safety Monitoring: Kontroler secara otomatis mendeteksi jika suhu melebihi batas kritis ($>70^{\circ}\text{C}$) dan langsung menyalakan alarm LED merah secara real-time.
3. Lightweight Cryptography: Mengamankan data hasil pembacaan sensor menggunakan operasi bitwise XOR sebelum ditransmisikan secara aman melalui komunikasi UART.

Kami juga menggunakan GNUPlot untuk melakukan visualisasi data pengujian, sehingga batasan antara zona SAFE (Aman) dan CRITICAL (Bahaya) dapat terpetakan dengan sangat jelas melalui grafik.

Dokumentasi teknis berupa simulasi proteus, flowchart, dan grafik hasil simulasi dapat dilihat pada lampiran di bawah ini.

#TeknikInstrumentasi

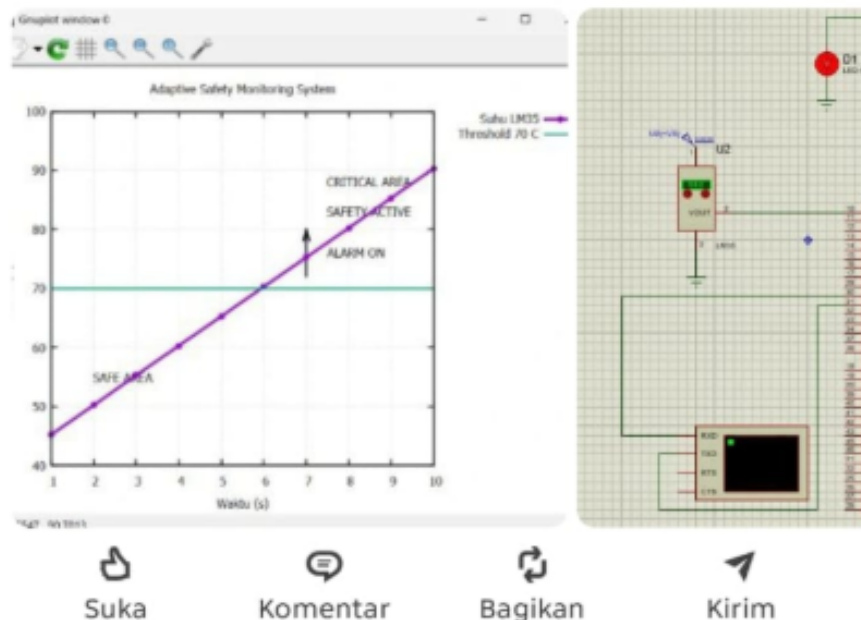


Figure 10: LinkedIn Documentation

Link:

https://www.linkedin.com/posts/angelenaexcel_teknikinstrumentasi-ugcPost-7463264829244424193-1PRz?utm_source=share&utm_medium=member_android&rcm=ACoAAFxv9woBBedoZi9Fr3bMJXiz5y_gpfr4PE

6 Conclusion

Based on the literature study of various modern embedded system journals, it can be concluded that current future work trends mainly focus on improving system security through Rust-based memory-safe programming, TinyML optimization on microcontrollers, adaptive safety monitoring, fault recovery mechanisms, lightweight cryptography, multicore scheduling, and optimization of communication and real-time data processing. Previous studies also indicate the increasing need for Artificial Intelligence (AI) integration in IoT-based embedded systems to improve adaptive monitoring and intelligent decision-making capabilities.

Based on these development trends, a new method called the Adaptive Safety-Aware Intelligent Embedded Controller Method was proposed by integrating various embedded system technologies into a single intelligent embedded controller architecture. The proposed method combines LM35 sensors, STM32 ADC processing, TinyML and AI processing, adaptive safety monitoring, lightweight cryptography, multicore scheduling, and UART communication to create a safer, more adaptive, and more efficient monitoring system for IoT and safety-critical applications.

Simulation results using Proteus, Keil uVision, and GNUPlot demonstrate that the system is capable of performing real-time temperature monitoring and detecting abnormal conditions automatically when the temperature exceeds the threshold value of 70°C. Under SAFE conditions, the monitoring system operates normally without activating alarms, while under CRITICAL conditions the system automatically activates LED alarms, buzzer notifications, and fault recovery mechanisms to prevent overheating in the embedded system. Furthermore, the GNUPlot visualization clearly illustrates the difference between SAFE AREA and CRITICAL AREA based on temperature changes during monitoring.

The main advantage of the proposed method lies in its ability to integrate multiple future work concepts from previous studies into one adaptive and intelligent embedded controller system. Compared to previous methods that only focused on individual aspects such as TinyML optimization, fault tolerance, or cloud monitoring separately, the proposed method combines AI processing, adaptive safety systems, secure communication, and real-time monitoring into one integrated architecture.

In addition, the proposed method has strong future development potential for industrial automation systems, IoT monitoring devices, smart instrumentation systems, and safety-

critical embedded applications. Future developments may include cloud-based monitoring integration, remote firmware updates, predictive maintenance systems, Edge AI optimization, and support for more advanced microcontroller architectures.

Overall, the Adaptive Safety-Aware Intelligent Embedded Controller Method successfully demonstrates an integrated embedded controller solution capable of providing real-time monitoring, adaptive safety protection, secure communication, and intelligent processing in modern embedded systems with limited computational resources.

DAFTAR PUSTAKA

- Carnelos, M., Pasti, F., & Bellotto, N. (2025). MicroFlow: An Efficient Rust-Based Inference Engine for TinyML. *Internet of Things*, 30. <https://doi.org/10.1016/j.iot.2025.101498>
- Cosimi, F., Arena, A., Saponara, S., & Gai, P. (2024). SLOPE: Safety Log Peripherals implementation and software drivers for a safe RISC-V microcontroller unit. *Microprocessors and Microsystems*, 110, 105103. <https://doi.org/10.1016/j.micpro.2024.105103>
- Dzulfiqar, F., Aprilia, L., Udhiarto, A., & Nuryadi, R. (2025). Development of a High-Sensitivity Microcontroller-Based Microcantilever Sensor System Using Direct Digital Synthesis Technology. *IEEE Transactions on Instrumentation and Measurement*, 74, 1–10. <https://doi.org/10.1109/TIM.2025.3566426>
- Frank, E., Schleiser, K., Fouquet, R., Zandberg, K., Amsuss, C., & Baccelli, E. (2025). Ariel OS: An Embedded Rust Operating System for Networked Sensors & Multi-Core Microcontrollers. *Proceedings - 2025 21st International Conference on Distributed Computing in Smart Systems and the Internet of Things*. <https://doi.org/10.1109/DCOSS-IoT65416.2025.00040>
- Kim, Y., Cho, J., Pyeon, Y. J., Kim, H., Lee, S., Kwak, J. H., Yoon, H., Lee, Y., Shin, H., & Kim, J. J. (2025). A Mixture-Gas Multisensor Interface With On-Chip Classification and On-Edge Regression. *IEEE Sensors Journal*, 25(16), 31435–31446. <https://doi.org/10.1109/JSEN.2025.3586060>
- Ma, Z., Liu, G., Eastman, A., Kaufman, K., Armanuzzaman, M., Tan, X., Jesse, K., Walls, R. J., & Zhao, Z. (2025). “We just did not have that on the embedded system”: Insights and Challenges for Securing Microcontroller Systems from the Embedded CTF Competitions. *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. <https://doi.org/10.1145/3719027.3765039>
- Mukherjee, N., Tille, D., Sapati, M., Liu, Y., Mayer, J., Milewski, S., Moghaddam, E., Rajski, J., Solecki, J., & Tyszer, J. (2021). Time and Area Optimized Testing of Automotive ICs. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 29(1), 76–88. <https://doi.org/10.1109/TVLSI.2020.3025138>
- Plaуска, I., Liutkevičius, A., & Janavičiūtė, A. (2023). Performance Evaluation of C/C++, MicroPython, Rust and TinyGo Programming Languages on ESP32

Microcontroller. *Electronics*, 12(1). <https://doi.org/10.3390/electronics12010143>

Rehman, A., Mahmood, K., Ul Hassan, M., Javed, M. W., Aurangzeb, K., & Anwar, M. S. (2025). LEMS: Optimized Large Model Framework for Edge-AI in Consumer Internet of Things Devices. *IEEE Transactions on Consumer Electronics*, 71(2), 5683–5690. <https://doi.org/10.1109/TCE.2025.3552533>

Samanta, A., Sharma, M., Locke, W., & Williamson, S. (2025). Cloud-Enhanced Battery Management System Architecture for Real-Time Data Visualization, Decision Making, and Remote Monitoring.

Sharma, A., Sharma, S., Tanksalkar, S. R., Torres-Arias, S., & Machiry, A. (2024). Rust for Embedded Systems: Current State and Open Problems.

Stahl, A., et al. (2020). Driver Generation for IoT Nodes using Optimized Runtime Communication.

Vandervelden, K., et al. (2024). Embedded Rust Frameworks for IoT and Networking Optimization.

Wang, P., et al. (2021). Fast Convolution Algorithm for ARM Cortex-M Microcontrollers.

Wu, Y., et al. (2021). FERNANDO Fault Tolerance Approach for Embedded Multicore Systems.

Zhou, L., et al. (2025). Real-Time Fault Recovery System for Adaptive Embedded Monitoring.